



ITI · AI-ASSISTED DEVELOPMENT TRACK

AI-Assisted Development & API Integration

Eng. Fatma Elraey

Lab 1 — Project Initialization & The AI Feature Sprint

Field	Detail
Instructor	Fatma Elraey
Course	AI-Assisted Development & API Integration (2-Day Course)
Session	Day 1 of 2 — Lab, 3 Hours
Focus	Setting up a real AI-assisted development environment and using it to build a complete feature from scratch, with a code-review mindset throughout.

Before You Start

You'll need Cursor (download from cursor.com) or VS Code with the GitHub Copilot extension installed, plus a small existing project to work in — any simple app boilerplate you have handy (a starter React app, a small Express API, or similar) works fine. If you don't have one ready, any "create-react-app"-style starter or a fresh git clone of a simple template is enough.

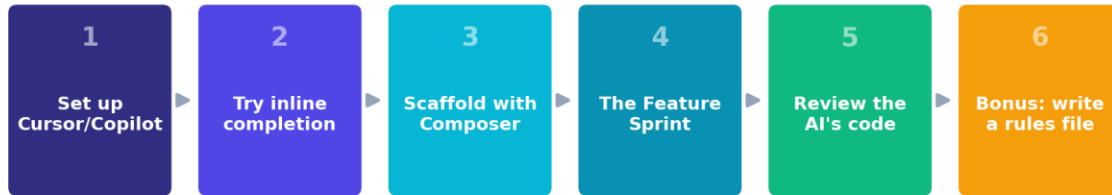


Figure L.1 — Six steps: set up, try each mode, sprint a feature, then review.

✓ QUICK CHECK — Before You Start

Q1. What are the three core AI-assistant interaction modes covered in the lecture?

Answer: *Inline completion, Chat, and Composer / Edit Mode.*

Q2. What's the point of a rules file like `.cursorrules`?

Answer: *It automatically supplies project context and conventions to every AI request in that project.*

Q3. What should you always do before accepting AI-generated code?

Answer: *Review it — read the diff, check for bugs and security issues, the same way you'd review a teammate's code.*



01

Project Initialization

Set up a clean AI-assisted development environment before touching any code.

- *Install Cursor, or install the GitHub Copilot extension in VS Code, and sign in.*
- *Open your existing simple project (or clone/create a small starter app) in the editor.*
- *Confirm the AI assistant is active: open a file and start typing — you should see inline ghost-text suggestions appear.*
- *Open the Chat panel and ask it: "Summarize what this project does, based on the files you can see." Confirm it can actually read your files.*

Why this matters: A five-minute setup check now saves a confusing hour later — the single most common early problem is the assistant not being signed in or not having the right folder open.

Try this:

CHAT PROMPT

Summarize what this project does, based on the files you can see.

02

Try Inline Completion

Get a feel for the lightest-touch mode before moving to bigger tasks.

- *Open any source file in your project and go to a blank line inside a function.*
- *Type a comment describing a small piece of logic (see the example below) and pause.*
- *Watch the ghost-text suggestion appear. Press Tab to accept it, or keep typing to ignore it.*
- *Try writing an incomplete function signature and see what the assistant suggests for the body.*

Why this matters: Comments describing intent are one of the most reliable ways to steer inline completion — the clearer the comment, the more useful the suggestion.

Try this:

```
// calculate the total price including 8% tax
```



03

Scaffold a New File with Composer

Practice the multi-file, whole-feature mode by scaffolding something small and self-contained.

- *Open Composer (Cursor) or Edit Mode (Copilot).*
- *Describe a small, self-contained new feature — see the starter prompt below for an example.*
- *Read every file the AI proposes before accepting anything. Note what it got right and anything you'd change.*
- *Ask a follow-up to adjust one detail (e.g. "use a different color scheme" or "add a loading state") and observe how it updates its own previous work.*

Why this matters: This is the exact same review habit from the lecture in practice — Composer output is a proposed change, not a finished change, until you've read it.

Try this:**COMPOSER PROMPT**

Create a new reusable Toast notification component. It should support 'success', 'error', and 'info' variants, auto-dismiss after 4 seconds, and match the existing styling conventions in this project.

04

The AI Feature Sprint

This is the main event. Take your existing simple app boilerplate and use the AI assistant to implement one complete, functional feature from scratch — not a toy snippet, a real working piece of the app.

- *Pick one feature to add: a simple authentication view (login form + a protected page), or a data filter/search box over an existing list of items. Either is a good scope for today.*
- *Use Composer / Edit Mode to describe the full feature in one clear, detailed prompt (see the framework below).*
- *Let the AI generate the code across whatever files it needs to touch.*
- *Read the full diff before accepting anything. Run the app and manually test the feature end to end.*
- *Fix or ask the AI to fix anything that doesn't work exactly as intended.*

Why this matters: This mirrors real professional work almost exactly: a clear feature request, an AI-generated first pass across multiple files, then human review and testing before it's considered done.

Prompt template:



FEATURE SPRINT PROMPT TEMPLATE

Add a [login view / search filter] to this project.

Requirements:

- [specific behavior #1]
- [specific behavior #2]
- Match the existing styling and file structure of this project.
- [any edge case worth handling, e.g. "show an error message on failed login"]



05

Review the AI's Code Like a Pull Request

Now put on your reviewer hat and formally check the feature you just built, using a short checklist instead of just "it looks fine."

- *Read through every changed file one more time, slowly.*
- *Check for hardcoded secrets or credentials anywhere in the new code.*
- *Check that any user input is handled safely (no unsanitized values passed directly into a query, a URL, or rendered HTML).*
- *Check naming and style consistency with the rest of the project.*
- *Write 2-3 sentences: what would you change if a teammate had submitted this as a pull request?*

Why this matters: *This checklist is small on purpose — it's meant to become a habit you run on every AI-generated change, not a one-time exercise.*



Bonus Stretch Goal — Write a Rules File

Write a real rules file for your project so every future prompt automatically includes the right context.

- Create `.cursorrules` (Cursor) or `.github/copilot-instructions.md` (Copilot) in your project root.
- Describe your stack, your conventions, and anything the AI should avoid (see the template below).
- Test it: open a fresh Chat or Composer session and ask for a small change without repeating any context. Confirm the assistant already knows your conventions.
- Optional: add one rule you wish the AI had followed during today's Feature Sprint, based on what you had to correct.

Why this matters: A well-written rules file is one of the highest-leverage five-minute investments in this whole workflow — you'll write it once and benefit from it on every prompt afterward.

Starter template:

```
# .copilot-instructions.md
This project uses [framework/language].
State management: [your approach].
Styling: [your approach].
Do NOT: [anything to avoid, e.g. "introduce
a new state library" or "use class components"].
Always: [a standing requirement, e.g. "write a
test for new utility functions"].
```

Nice work! Tomorrow you'll move from using AI tools to building with AI directly — calling LLM APIs yourself.